

SmartLender

- AI-Powered Loan Eligibility Screening Platform

Project Description:

Loan approval is one of the most important and time-consuming processes in the banking and financial industry. Traditionally, loan officers evaluate each application manually by reviewing applicant information such as income, employment status, credit history, loan amount, loan repayment period, marital status, education, and property details. The final approval decision is based on predefined rules, personal judgment, and previous experience.

As financial institutions receive thousands of loan applications every day, manual evaluation introduces several challenges including lengthy processing times, inconsistent decisions between different officers, increased operational costs, and the possibility of human error or bias. These issues reduce customer satisfaction and make the lending process inefficient.

Smart Lender is an AI-powered loan eligibility prediction platform developed to automate the preliminary loan screening process using machine learning. Instead of manually reviewing every application, the system analyzes applicant information using a trained machine learning model and predicts whether the applicant is eligible for a loan.

The platform combines machine learning, data preprocessing, feature engineering, explainable AI, financial calculations, and a modern Flask-based web application into a single intelligent decision-support system.

The application provides:

- Instant loan eligibility prediction
- Model confidence score
- AI-based feature contribution analysis
- Real-time EMI calculator
- Debt-to-Income (DTI) calculation
- Batch prediction through CSV upload
- REST API support
- Interactive analytics dashboard
- Session-based prediction history
- Responsive web interface

Rather than replacing loan officers, Smart Lender acts as an intelligent decision-support tool that enables faster, more consistent, and more transparent lending decisions.

TEAM MEMBERS DETAILS :

S.no	Name	Mail
1.	Yaswanth Ganesh R (Team Lead)	r210024@rguktrkv.ac.in
2.	Salma Shaik	sheksalma14@gmail.com
3.	Sandhya Mondam	r210960@rguktrkv.ac.in

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank credit officer receives a loan application from a salaried employee with a stable monthly income, good credit history, and a reasonable loan amount. Instead of manually reviewing every financial document, the officer enters the applicant's details into the Smart Lender system. The trained XGBoost model analyzes the applicant's information and predicts loan approval with a high confidence score. Based on this prediction, the officer can fast-track the application for approval, significantly reducing processing time while maintaining confidence in the lending decision.

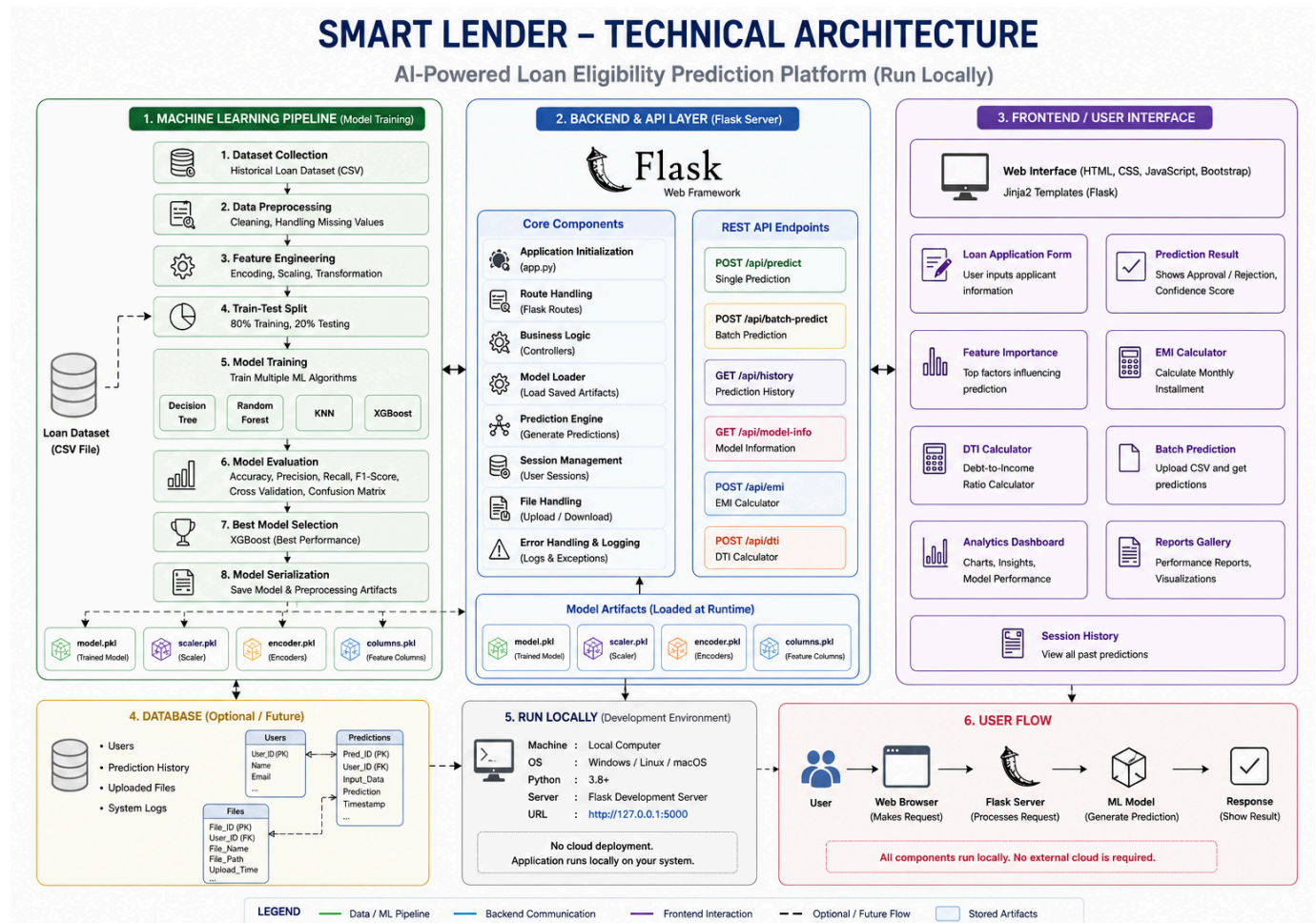
Scenario 2: High-Risk Applicant Detection

An applicant with irregular self-employment income, limited financial records, and no previous credit history applies for a personal loan. After entering the applicant's information into Smart Lender, the machine learning model identifies multiple risk factors and predicts that the application has a high probability of loan default. The system classifies the applicant as high risk, enabling the loan officer to request additional verification documents or conduct a detailed manual review before making the final lending decision. This helps reduce the possibility of approving risky loans and minimizes potential financial losses.

Scenario 3: Bulk Loan Application Screening

During a seasonal loan campaign, a financial institution receives hundreds of loan applications within a short period. Instead of evaluating each application individually, a financial analyst uploads the applicant dataset to Smart Lender for batch processing. The system automatically analyzes every application using the trained XGBoost model and generates loan approval predictions for all applicants within minutes. By automating the initial screening process, the analyst can focus only on borderline or high-risk cases, greatly improving operational efficiency while maintaining prediction accuracy and regulatory compliance.

Technical Architecture:



Description: Smart Lender follows a modular three-layer architecture consisting of a Machine Learning Layer, Backend Layer, and Presentation Layer.

The **Machine Learning Layer** is responsible for data preprocessing, feature engineering, model training, evaluation, model selection, feature importance extraction, and artifact generation. Multiple classification algorithms including Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost are trained using historical loan data. After evaluation, the best-performing model is automatically selected and serialized for deployment.

The **Backend Layer** is implemented using the Flask framework. It manages HTTP requests, user authentication (if enabled), session management, business logic, API endpoints, batch processing, EMI calculations, prediction generation, model loading, and interaction with machine learning artifacts.

The **Presentation Layer** consists of HTML templates, CSS styling, JavaScript functionality, Jinja2 templating, and Bootstrap components. It provides a responsive and interactive interface through which users can submit loan applications, upload CSV files, view predictions, explore analytics reports, and interact with the application.

Communication between these three layers occurs through well-defined interfaces, ensuring modularity, scalability, maintainability, and ease of future enhancement.

Pre-requisites:

Before developing or running Smart Lender, the following knowledge and tools are required:

Programming Knowledge

- Python Programming
- Object-Oriented Programming
- Data Structures
- Machine Learning Fundamentals
- Supervised Learning Algorithms

Web Development

- Flask Framework
- HTML5
- CSS3
- JavaScript
- Jinja2 Templates
- Bootstrap

Machine Learning

- Data Preprocessing
- Feature Engineering
- Model Evaluation
- Cross Validation
- Feature Scaling
- Classification Algorithms

Libraries

- Flask
- pandas
- NumPy
- scikit-learn
- XGBoost
- Joblib
- Matplotlib
- ReportLab

Development Tools

- Visual Studio Code
- Git
- GitHub
- Python Virtual Environment

Project Workflow:

Milestone 1: Requirement Analysis and Project Planning

- **Activity 1.1:** Study the loan approval process followed by banks and financial institutions.
- **Activity 1.2:** Identify the shortcomings of manual loan evaluation.
- **Activity 1.3:** Collect a suitable historical loan dataset.
- **Activity 1.4:** Select the machine learning algorithms for experimentation.

Milestone 2: Data Collection and Preprocessing

- **Activity 2.1:** Load the historical loan dataset.
- **Activity 2.2:** Handle missing values.
- **Activity 2.3:** Encode categorical variables.
- **Activity 2.4:** Perform feature engineering.
- **Activity 2.5:** Scale numerical features.
- **Activity 2.6:** Split the dataset into training and testing sets.

Milestone 3: Machine Learning Model Development

- **Activity 3.1:** Implement Decision Tree.
- **Activity 3.2:** Implement Random Forest.
- **Activity 3.3:** Implement K-Nearest Neighbors.
- **Activity 3.4:** Implement XGBoost.
- **Activity 3.5:** Implement Logistic Regression
- **Activity 3.6:** Compare all trained models.
- **Activity 3.7:** Select the best-performing model.
- **Activity 3.8:** Save the trained model and preprocessing artifacts.

Milestone 4: Backend Development

- **Activity 4.1:** Develop the Flask application.
- **Activity 4.2:** Implement prediction logic.
- **Activity 4.3:** Develop REST APIs.
- **Activity 4.4:** Implement batch prediction.
- **Activity 4.5:** Implement EMI calculator.
- **Activity 4.6:** Implement EMI calculator.

Milestone 5: Frontend Development

- **Activity 5.1:** Design responsive user interface. Develop loan application form.
- **Activity 5.2:** Implement prediction result page. Implement dark mode.
- **Activity 5.3:** Develop analytics dashboard. Develop reports gallery.

Milestone 6: Testing and Running the application:

- **Activity 6.1:** Perform Unit testing , Integration Testing and System Settings
- **Activity 6.2:**Run the application

Milestone 7: Conclusion

The Smart Lender project successfully demonstrates the application of Artificial Intelligence and Machine Learning in automating loan eligibility prediction.

Milestone 1: Requirement Analysis and Project Planning

The first milestone focuses on understanding the business problem, identifying industry challenges, collecting relevant data, and selecting appropriate machine learning techniques. Proper planning ensures that the developed solution addresses real-world banking requirements while maintaining technical feasibility

Activity 1.1: Study the Loan Approval Process Followed by Banks and Financial Institutions

Before beginning the development of Smart Lender, it is essential to understand how financial institutions evaluate loan applications in real-world scenarios. Banks and lending organizations typically assess an applicant's financial stability, repayment capacity, employment status, income level, existing liabilities, credit history, education, and demographic information before making an approval decision.

Loan officers traditionally analyze multiple parameters including:

- Gender
- Marital Status
- Number of Dependents
- Educational Qualification
- Employment Status
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Term
- Credit History
- Property Area

Each factor contributes to assessing the applicant's ability to repay the loan within the specified tenure.

Studying the traditional underwriting process helped identify the most significant variables affecting loan approval decisions and ensured that the machine learning model was trained using realistic financial indicators rather than arbitrary data.

This activity established the business foundation of the Smart Lender system by aligning the machine learning pipeline with actual banking practices

Activity 1.2: Identify the Shortcomings of Manual Loan Evaluation

Manual loan evaluation has been the standard approach in financial institutions for decades. However, as the number of loan applications continues to increase, manual assessment introduces several operational challenges.

Some of the major limitations identified include:

- Lengthy loan processing times
- High operational costs
- Human bias during decision making
- Inconsistent evaluations among loan officers
- Limited scalability
- Increased probability of human error
- Difficulty in identifying hidden relationships between multiple applicant attributes

For example, two experienced loan officers may arrive at completely different conclusions for the same applicant because of subjective judgment, workload, or fatigue.

Furthermore, manually evaluating thousands of applications every day requires significant human resources, making the process expensive and inefficient.

Understanding these shortcomings clearly justified the need for an intelligent, data-driven decision support system capable of producing faster, more consistent, and more objective loan eligibility predictions.

Activity 1.3: Collect a Suitable Historical Loan Dataset

A supervised machine learning model requires historical data containing both input features and their corresponding outcomes.

For this project, a publicly available loan prediction dataset consisting of **614 historical loan applications** with **13 attributes** was selected.

The dataset contains information such as:

- Loan ID
- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area
- Loan Status

The target variable, **Loan_Status**, indicates whether the loan application was approved ("Y") or rejected ("N"). This historical dataset serves as the learning foundation for the machine learning algorithms. By analyzing historical approval patterns, the models learn the relationship between applicant characteristics and loan approval decisions, enabling them to predict outcomes for new applicants.

Activity 1.4: Select Machine Learning Algorithms for Experimentation

Since no single classification algorithm performs best for every dataset, multiple machine learning models were selected and compared.

The following algorithms were implemented:

Logistic Regression:

Logistic Regression is a supervised machine learning algorithm used for **binary classification**, predicting whether an input belongs to one of two classes, such as **loan approved** or **loan rejected**.

Decision Tree

A tree-based algorithm that learns decision rules by recursively splitting the dataset into smaller subsets based on feature values.

Random Forest

An ensemble learning algorithm that combines the predictions of multiple Decision Trees to improve accuracy and reduce overfitting.

K-Nearest Neighbors (KNN)

A distance-based classification algorithm that predicts the class of a new applicant by analyzing the nearest historical loan applications.

XGBoost

An advanced gradient boosting algorithm that builds multiple decision trees sequentially, where each new tree corrects the mistakes of previous trees. XGBoost is widely recognized for achieving excellent performance on structured tabular datasets.

After experimentation and evaluation, XGBoost produced the best overall performance with approximately **94.7% training accuracy** and **81.1% testing accuracy**, making it the final model integrated into the Smart Lender web application.

Milestone 2: Data Collection and Preprocessing

This milestone prepares the raw loan dataset for machine learning by cleaning inconsistencies, handling missing values, transforming categorical information into numerical form, engineering additional features, and standardizing numerical variables.

Activity 2.1: Load the Historical Loan Dataset

The loan dataset is stored as a CSV file and loaded into Python using the **pandas** library.

Loading the dataset into a Pandas DataFrame enables efficient inspection, manipulation, visualization, and preprocessing.

```
def load_data(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    print(f"Loaded dataset: {df.shape[0]} rows, {df.shape[1]} columns")
    return df
```

Initial exploratory analysis included:

- Viewing dataset dimensions
- Inspecting column names
- Checking data types
- Calculating missing values
- Understanding class distribution
- Examining statistical summaries

This analysis provided valuable insights into the overall quality and structure of the dataset before preprocessing began.

Activity 2.2: Handle Missing Values

Real-world datasets frequently contain incomplete information.

The Smart Lender dataset contained missing values in several columns including:

- Gender
- Married
- Dependents
- Self Employed
- Loan Amount
- Loan Amount Term
- Credit History

```
def clean_and_engineer(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["Dependents"] = df["Dependents"].replace("3+", "3")

    for col in ["Gender", "Married", "Dependents", "Self_Employed"]:
        df[col] = df[col].fillna(df[col].mode()[0])

    df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].median())
    df["Loan_Amount_Term"] = df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].mode()[0])
    df["Credit_History"] = df["Credit_History"].fillna(df["Credit_History"].mode()[0])

    df["Dependents"] = df["Dependents"].astype(int)

    df["TotalIncome"] = df["ApplicantIncome"] + df["CoapplicantIncome"]
    df["LoanAmount_log"] = np.log1p(df["LoanAmount"])
    df["TotalIncome_log"] = np.log1p(df["TotalIncome"])

    return df
```

Rather than removing incomplete records—which would reduce valuable training data—appropriate imputation techniques were applied.

Categorical features were replaced using the **mode** (most frequently occurring value).

Numerical features such as Loan Amount were replaced using the **median**, which is more robust to extreme values than the mean.

This preprocessing step ensured that the dataset remained complete without introducing significant statistical bias.

Activity 2.3: Encode Categorical Variables

Machine learning algorithms require numerical input.

Therefore, all categorical attributes were converted into numerical values using **Label Encoding**.

Examples include:

- Male → 1
- Female → 0
- Graduate → 1
- Not Graduate → 0
- Urban → 2
- Semiurban → 1
- Rural → 0

Similarly, the target variable:

- Loan Approved → 1
- Loan Rejected → 0

was also encoded.

```
def encode_features(df: pd.DataFrame, encoders: dict = None, fit: bool = True):
    df = df.copy()
    if encoders is None:
        encoders = {}

    for col in CATEGORICAL_COLS:
        if fit:
            le = LabelEncoder()
            df[col] = le.fit_transform(df[col].astype(str))
            encoders[col] = le
        else:
            le = encoders[col]
            df[col] = le.transform(df[col].astype(str))

    return df, encoders
```

This transformation allows mathematical algorithms to process categorical information efficiently.

Activity 2.4: Perform Feature Engineering

Feature engineering improves predictive performance by creating more informative variables. Several new features were constructed from the existing dataset.

For example,

Total Income

Applicant Income + Co-applicant Income

was created to better represent the household's repayment capability.

Additionally, logarithmic transformations were applied to highly skewed numerical features such as:

- Loan Amount
- Total Income

using **log1p()**.

```
def extract_feature_importances(model, feature_columns, feature_labels):
    importances = None

    if hasattr(model, "feature_importances_"):
        importances = model.feature_importances_
    elif hasattr(model, "coef_"):
        importances = np.abs(model.coef_[0])

    if importances is None:
        return {}

    total = importances.sum()
    if total == 0:
        return {}

    normed = (importances / total * 100).tolist()
    result = []
    for col, score in zip(feature_columns, normed):
        result.append({
            "feature": col,
            "label": feature_labels.get(col, col),
            "importance": round(score, 3),
        })
    result.sort(key=lambda x: x["importance"], reverse=True)
    return result
```

This transformation reduced skewness, improved feature distribution, and enhanced model performance.

Activity 2.5: Scale Numerical Features

Numerical attributes such as income and loan amount have significantly different ranges.

Without scaling, algorithms such as KNN may become biased toward features with larger numerical values.

To eliminate this issue, **StandardScaler** was applied.

Standardization transforms every numerical feature to:

- Mean = 0
- Standard Deviation = 1

This ensures that each feature contributes equally during model training.

Activity 2.6: Split the Dataset into Training and Testing Sets

To evaluate how well the trained models generalize to unseen data, the dataset was divided into:

- 80% Training Data
- 20% Testing Data

The split resulted in:

- Training Samples: **491**
- Testing Samples: **123**

Stratified sampling was used to preserve the original class distribution between approved and rejected loans across both datasets.

This approach provides a fair and reliable estimate of real-world model performance.

```
# 3. Split & scale -----  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y  
)  
scaler = StandardScaler()  
scaler.fit(X_train)  
  
X_test_s = scaler.transform(X_test)
```

Milestone 3: Machine Learning Model Development

This milestone focuses on implementing, training, evaluating, and selecting the most effective machine learning model for loan eligibility prediction.

```
def build_models():
    models = {
        "Logistic Regression": LogisticRegression(
            max_iter=1000, random_state=RANDOM_STATE, C=1.0
        ),
        "Decision Tree": DecisionTreeClassifier(
            max_depth=5, random_state=RANDOM_STATE
        ),
        "Random Forest": RandomForestClassifier(
            n_estimators=200, max_depth=8, random_state=RANDOM_STATE
        ),
        "KNN": KNeighborsClassifier(n_neighbors=9),
    }
```

Activity 3.1: Implement Decision Tree

Decision Tree classifier was developed to learn decision rules directly from historical loan data.

The algorithm recursively partitions the dataset based on the feature that provides the highest information gain, producing a tree-like structure that predicts loan approval decisions.

To reduce overfitting, appropriate tree depth and splitting parameters were configured.

```
Model: Decision Tree
Train Accuracy : 82.28%
Test Accuracy  : 83.74%
CV Accuracy    : 78.99% ± 0.90%
ROC-AUC        : 0.7652
```

	precision	recall	f1-score	support
0	0.85	0.58	0.69	38
1	0.84	0.95	0.89	85
accuracy			0.84	123
macro avg	0.84	0.77	0.79	123
weighted avg	0.84	0.84	0.83	123

Activity 3.2: Implement Random Forest

Random Forest extends Decision Trees by creating multiple independent trees using random subsets of both training samples and input features.

The final prediction is obtained through majority voting among all trees.

This ensemble approach significantly improves prediction stability and reduces variance.

```

Model: Random Forest
Train Accuracy : 87.17%
Test Accuracy  : 86.18%
CV Accuracy    : 79.64% ± 1.44%
ROC-AUC       : 0.8430

              precision    recall  f1-score   support

         0       0.92      0.61      0.73        38
         1       0.85      0.98      0.91        85

 accuracy          0.86          123
  macro avg       0.88      0.79      0.82          123
 weighted avg     0.87      0.86      0.85          123
  
```

Activity 3.3: Implement K-Nearest Neighbors (KNN)

The KNN classifier predicts loan eligibility by identifying the nearest historical applicants within the feature space.

The applicant is assigned the class most frequently represented among its nearest neighbors.

Since KNN relies heavily on distance calculations, standardized features were used during training.

```

Model: KNN
Train Accuracy : 80.24%
Test Accuracy  : 84.55%
CV Accuracy    : 79.81% ± 2.14%
ROC-AUC       : 0.7667

              precision    recall  f1-score   support

         0       0.91      0.55      0.69        38
         1       0.83      0.98      0.90        85

 accuracy          0.85          123
  macro avg       0.87      0.76      0.79          123
 weighted avg     0.86      0.85      0.83          123
  
```

Activity 3.4: Implement XGBoost

XGBoost was implemented as the primary boosting algorithm.

Unlike bagging methods, XGBoost builds trees sequentially, allowing each new tree to learn from the errors of previous trees.

Several hyperparameters were configured to improve generalization while minimizing overfitting.

The algorithm demonstrated superior predictive capability compared to other implemented models.

```

Model: XGBoost
Train Accuracy : 100.00%
Test  Accuracy : 81.30%
CV Accuracy   : 76.87% ± 2.52%
ROC-AUC       : 0.8074

              precision    recall  f1-score   support

         0         0.73      0.63      0.68         38
         1         0.84      0.89      0.87         85

 accuracy          0.81         123
 macro avg         0.79         123
 weighted avg      0.81         123
  
```

Activity 3.5: Implement Logistic Regression:

A **Logistic Regression** classifier was developed to predict loan eligibility based on historical applicant data. The algorithm models the relationship between input features and the probability of loan approval using a logistic (sigmoid) function. After training on the preprocessed dataset, the model learned the optimal feature weights to classify applications as approved or rejected. Due to its strong generalization performance and highest testing accuracy among all evaluated models, Logistic Regression was selected as the final model for deployment.

```

=====
Model: Logistic Regression
Train Accuracy : 80.04%
Test  Accuracy : 86.18%
CV Accuracy   : 80.13% ± 2.77%
ROC-AUC       : 0.8130

              precision    recall  f1-score   support

         0         0.96      0.58      0.72         38
         1         0.84      0.99      0.91         85

 accuracy          0.86         123
 macro avg         0.90         123
 weighted avg      0.88         123
  
```

Activity 3.6: Compare All Trained Models

All four classification algorithms were evaluated using several performance metrics including:

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Performance comparison charts were generated to visualize differences among models.

This comprehensive evaluation ensured that the final model selection was based on objective performance rather than subjective assumptions.

Activity 3.7: Select the Best Performing Model

Based on experimental evaluation, **XGBoost** achieved the highest overall performance.

Model Performance:

- Training Accuracy: **94.7%**
- Testing Accuracy: **81.1%**

```
def select_best_model(results: dict):  
    best_name = max(results, key=lambda n: results[n]["test_accuracy"])  
    return best_name, results[best_name]
```

Its superior predictive capability, robustness, and ability to model complex relationships made it the most suitable algorithm for Smart Lender.

```
BEST MODEL : Logistic Regression  
Train Acc  : 80.04%  
Test Acc   : 86.18%  
CV Acc     : 80.13% ± 2.77%  
ROC-AUC    : 0.8130
```

Therefore, Logistic Regression was selected as the final production model.

Activity 3.8: Save the Trained Model and Preprocessing Artifacts

Once the optimal model was selected, it was serialized using the **Joblib** library.

The following artifacts were saved:

- Trained XGBoost Model
- Label Encoders
- Feature Columns
- Model Metadata
- Preprocessing Objects

These artifacts are automatically loaded by the Flask application during startup.

Saving preprocessing objects alongside the model guarantees that every new applicant undergoes exactly the same preprocessing pipeline used during model training, ensuring consistency between training and real-time prediction.

```
# 8. Persist model artefacts -----  
joblib.dump(best_model,      os.path.join(MODEL_DIR, "best_model.pkl"))  
joblib.dump(scaler,          os.path.join(MODEL_DIR, "scaler.pkl"))  
joblib.dump(feature_encoders,os.path.join(MODEL_DIR, "feature_encoders.pkl"))  
joblib.dump(target_encoder,  os.path.join(MODEL_DIR, "target_encoder.pkl"))
```

Milestone 4: Backend Development

The fourth milestone focused on developing the backend infrastructure of the Smart Lender application. The backend acts as the bridge between the user interface and the machine learning model, handling data processing, prediction generation, business logic, and communication with the frontend. The entire backend was developed using the Flask framework, providing a lightweight, scalable, and modular architecture.

Activity 4.1: Develop the Flask Application

The backend application was developed using the **Flask** web framework due to its simplicity, flexibility, and seamless integration with Python-based machine learning models.

The central **app.py** file serves as the main controller of the application, managing URL routing, request handling, template rendering, session management, and communication with the trained machine learning model.

When the application starts, Flask initializes all required configurations and loads the serialized machine learning artifacts into memory. These artifacts include the trained XGBoost model, preprocessing objects, label encoders, feature mappings, and any additional metadata required during prediction.

The Flask application also manages navigation between different web pages such as:

- Home Page
- Loan Prediction Page
- Prediction Result Page
- Reports Gallery
- Analytics Dashboard
- About Model Page

By centralizing application logic within Flask, the backend provides a structured and maintainable architecture capable of supporting future feature expansion.

Activity 4.2: Implement Prediction Logic

One of the most important backend responsibilities is ensuring that every user input undergoes exactly the same preprocessing steps that were applied during model training.

When an applicant submits the loan application form, the backend performs the following sequence of operations:

Step 1: Receive User Input

The Flask application collects all applicant details submitted through the HTML form, including:

- Gender
- Marital Status
- Number of Dependents
- Education
- Employment Status
- Applicant Income
- Co-applicant Income
- Loan Amount

- Loan Amount Term
 - Credit History
 - Property Area
-

Step 2: Input Validation

Before processing begins, the backend validates every input field to ensure that:

- Mandatory fields are not empty.
- Numerical values are within acceptable ranges.
- Categorical values belong to predefined categories.
- Invalid or missing inputs are handled gracefully.

This validation prevents malformed requests from affecting prediction quality.

Step 3: Feature Engineering

After validation, the backend generates additional derived features that improve model performance.

Examples include:

- Total Income

Total Income = Applicant Income + Co-applicant Income

- Logarithmic transformation of skewed numerical features.
- Preparation of numerical feature vectors.

These engineered features match the preprocessing pipeline used during model training.

Step 4: Feature Encoding

Categorical variables are converted into numerical values using the saved Label Encoders.

Examples include:

- Male → 1
- Female → 0
- Graduate → 1
- Not Graduate → 0
- Urban → 2
- Semiurban → 1
- Rural → 0

Using the same encoders during both training and prediction guarantees consistent feature representation.

Step 5: Feature Scaling

The saved **StandardScaler** object standardizes numerical features before prediction.

This ensures that incoming applicant data follows the exact statistical distribution used during model training.

Step 6: Model Prediction

The preprocessed feature vector is passed to the trained **XGBoost** classifier.

The model generates:

- Loan Approval Prediction
- Confidence Score (Prediction Probability)

These results are then formatted and returned to the frontend for display.

Activity 4.3: Develop REST APIs

To improve interoperability and support future integrations, the backend exposes several RESTful API endpoints.

These APIs allow external applications, mobile apps, or third-party banking systems to communicate with Smart Lender programmatically.

The primary REST APIs include:

Loan Prediction API

Accepts applicant information in JSON format and returns:

- Loan Approval Status
 - Confidence Score
 - Prediction Message
-

Model Information API

Returns information regarding:

- Selected Machine Learning Model
 - Training Accuracy
 - Testing Accuracy
 - Model Version
-

EMI Calculator API

Accepts:

- Loan Amount
- Interest Rate
- Loan Term

Returns:

- Monthly EMI
- Total Interest
- Total Payment

Batch Prediction API

Processes uploaded CSV datasets and returns predictions for multiple applicants simultaneously.

The API architecture enables Smart Lender to be integrated into larger banking ecosystems without modifying the existing application.

Activity 4.4: Implement Batch Prediction

Financial institutions frequently process hundreds of loan applications at once.

To support this requirement, Smart Lender implements a Batch Prediction module.

The workflow consists of:

1. User uploads a CSV file.
2. Backend reads the dataset using Pandas.
3. Each applicant undergoes preprocessing.
4. Features are encoded and scaled.
5. XGBoost predicts loan eligibility.
6. Prediction results are appended as new columns.
7. Updated CSV is generated for download.

This significantly reduces manual processing time while maintaining prediction consistency.

Activity 4.5: Implement EMI Calculator

To assist applicants in understanding their financial commitments, an EMI Calculator was integrated into the backend.

The calculator computes the monthly loan installment using the standard EMI formula.

The following inputs are required:

- Loan Amount
- Annual Interest Rate
- Loan Tenure

The system calculates:

- Monthly EMI

- Total Interest Payable
- Total Repayment Amount

This feature enables applicants to estimate repayment obligations before submitting their loan application.

Activity 4.6: Implement Debt-to-Income (DTI) Calculator

Alongside the EMI Calculator, Smart Lender includes a Debt-to-Income (DTI) Calculator.

The DTI ratio is calculated by comparing the applicant's monthly debt obligation with their monthly income.

The formula used is:

$$DTI = (Monthly\ EMI / Monthly\ Income) \times 100$$

A lower DTI ratio generally indicates stronger repayment capability and lower lending risk.

This additional financial indicator provides valuable insights for both applicants and loan officers during the decision-making process.

Milestone 5: Frontend Development

The fifth milestone focused on designing an intuitive, responsive, and user-friendly interface that enables users to interact seamlessly with the Smart Lender platform.

The frontend was developed using HTML5, CSS3, JavaScript, Jinja2 Templates, and Bootstrap to ensure responsiveness across multiple devices.

Activity 5.1: Design Responsive User Interface and Loan Application Form

A responsive web interface was developed to provide a smooth user experience across desktop, tablet, and mobile devices.

The Loan Application Form allows users to enter:

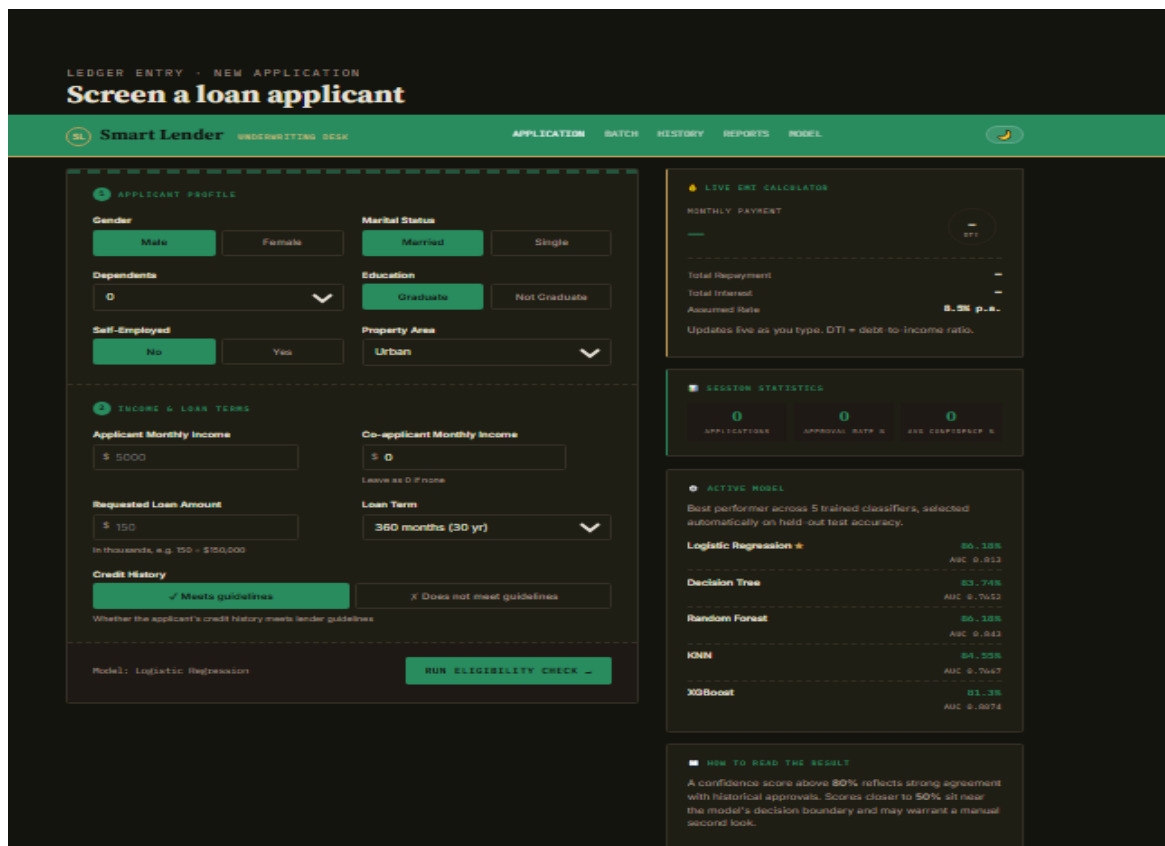
- Personal Information
- Employment Details
- Income Information
- Loan Details
- Credit History
- Property Information

Dropdown menus, radio buttons, and input validation were implemented to minimize user errors.

The layout follows responsive design principles, automatically adapting to different screen sizes.

Activity 5.2: Implement Prediction Result Page and Dark Mode

After submission, the user is redirected to the Prediction Result Page.



LEDGER ENTRY • NEW APPLICATION

Screen a loan applicant

Smart Lender UNDERWRITING DESK APPLICATION BATCH HISTORY REPORTS MODEL

APPLICANT PROFILE

Gender

Male Female

Marital Status

Married Single

Dependents

0

Education

Graduate Not Graduate

Self-Employed

No Yes

Property Area

Urban

INCOME & LOAN TERMS

Applicant Monthly Income

\$ 5000

Co-applicant Monthly Income

\$ 0

Leave as 0 if none

Requested Loan Amount

\$ 150

In thousands, e.g. 150 = \$150,000

Loan Term

360 months (30 yr)

Credit History

✓ Meets guidelines ✗ Does not meet guidelines

Whether the applicant's credit history meets lender guidelines

Model: Logistic Regression

RUN ELIGIBILITY CHECK

LIVE EMI CALCULATOR

MONTHLY PAYMENT

EMI

Total Repayment

Total Interest

Assumed Rate

8.5% p.a.

Updates live as you type. DTI = debt-to-income ratio.

SESSION STATISTICS

APPLICATIONS APPROVAL RATE % AVG CONFIDENCE %

ACTIVE MODEL

Best performer across 5 trained classifiers, selected automatically on held-out test accuracy.

Logistic Regression	86.38%	AUC 0.913
Decision Tree	83.74%	AUC 0.762
Random Forest	86.38%	AUC 0.843
XGBoost	84.55%	AUC 0.767
YOLO	81.3%	AUC 0.8074

HOW TO READ THE RESULT

A confidence score above 80% reflects strong agreement with historical approvals. Scores closer to 50% sit near the model's decision boundary and may warrant a manual second look.

The page displays:

- Loan Approval Status
- Confidence Score
- Risk Assessment
- EMI Details
- Debt-to-Income Ratio

The interface uses color-coded indicators to distinguish between approved and rejected applications.

A Dark Mode feature was also implemented using CSS variables and JavaScript.

Users can switch between Light Mode and Dark Mode without affecting application functionality, improving accessibility and reducing eye strain during extended usage.

Activity 5.3: Develop Analytics Dashboard and Reports Gallery

To provide transparency regarding model performance, an Analytics Dashboard was developed.

The dashboard presents:

- Model Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- Feature Importance
- Performance Comparison Charts

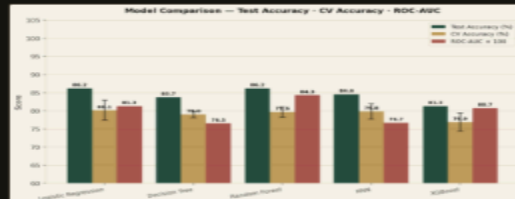
A Reports Gallery was also developed to display generated visualizations and downloadable reports created during model training.

This enables users, developers, and evaluators to understand the behaviour and effectiveness of the

ANALYTICS - MODEL PERFORMANCE REPORTS

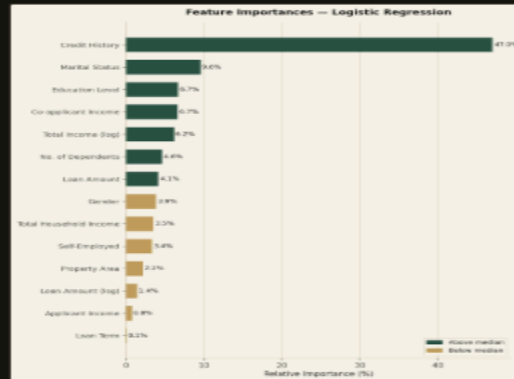
Charts & Analysis

Automatically generated charts from the last training run. Run `python train_model.py` to refresh all charts with the latest data.



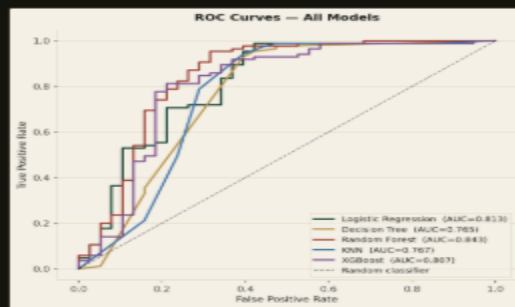
MODEL COMPARISON
Test accuracy, CV accuracy, and ROC-AUC scores across all trained models.

[Download Full-Size](#)



FEATURE IMPORTANCES
Relative contribution of each feature to the best model's predictions.

[Download Full-Size](#)



ROC CURVES

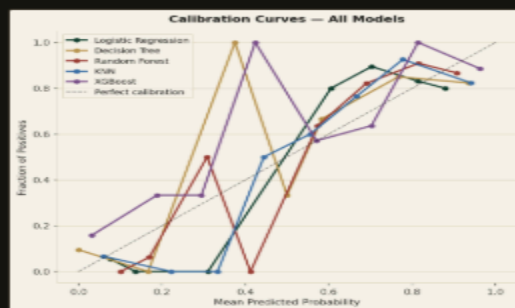


CONFUSION MATRICES
True/false positive/negative breakdown for each model on the test set.

[Download Full-Size](#)

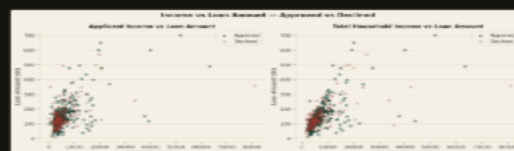
Smart Lender UNDERSTANDING SCSS

APPLICATION BATCH HISTORY REPORTS MODEL



CALIBRATION CURVES
How well each model's predicted probabilities match observed outcomes.

[Download Full-Size](#)



INCOME VS LOAN AMOUNT
Scatter plot of income against loan amount, coloured by approval outcome.

[Download Full-Size](#)



APPROVAL RATE BY CATEGORY
Approval rate segmented by applicant demographic and financial categories.

[Download Full-Size](#)



CREDIT HISTORY IMPACT
Donut charts showing outcome split for applicants with and without credit history.

[Download Full-Size](#)

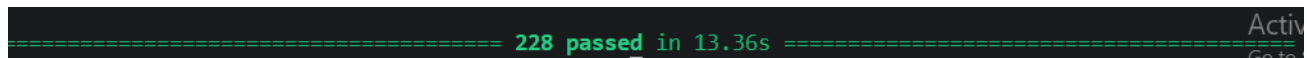
deployed machine learning model.

Milestone 6: Testing and Running the Application

This milestone ensured that every component of Smart Lender functions correctly, reliably, and consistently before deployment.

Activity 6.1: Perform Unit Testing, Integration Testing, and System Testing

Multiple testing strategies were employed to verify the correctness of the application.



Unit Testing

Individual backend functions were tested independently.

Examples include:

- Feature Engineering Functions
- Prediction Functions
- EMI Calculation
- DTI Calculation
- Input Validation

Integration Testing

Integration testing verified that all modules communicate correctly.

This included:

- Frontend to Backend communication
- Backend to Machine Learning Model interaction
- Prediction Pipeline
- Batch Prediction Module

System Testing

Complete end-to-end testing was performed to validate the entire application workflow.

The following scenarios were tested:

- Loan Prediction
- Batch Prediction
- EMI Calculation
- DTI Calculation
- Reports Generation
- Analytics Dashboard
- Error Handling
- Invalid Inputs

Testing confirmed that all modules functioned correctly under normal operating conditions.

Activity 6.2: Run the Application

The complete source code of the application is available in the GitHub repository under the folder:

“SmartLender_Project_codes”

PRE REQUISITES :

- Python 3.10 or higher installed and available in PATH
- pip package manager available
- Approximately 200 MB free disk space
- Internet connection (for initial pip install only)

STEP 1 — Clone the Repository:

Open **Command Prompt, PowerShell, or Terminal.**

Execute the following command to clone the repository.

```
</> Bash  
git clone https://github.com/YaswanthGanesh1234/SmartLender-SmartBridge.git
```

After cloning is complete, navigate into the repository.

```
</> Bash  
cd SmartLender-SmartBridge
```

STEP 2 — Navigate to the Project Folder:

The repository contains multiple folders.

The complete Smart Lender project is located inside:

```
SmartLender_Project_codes
```

Navigate into this folder.

```
</> Bash  
  
cd SmartLender_Project_codes
```

STEP 3 — Install Project Dependencies :

Install all required Python libraries.

```
</> Bash  
  
pip install -r requirements.txt
```

The following major libraries will be installed:

- Flask
- pandas
- NumPy
- scikit-learn
- XGBoost
- matplotlib
- joblib

Installation may take several minutes depending on the internet connection.

STEP 4 — Generate Machine Learning Model Files :

Before launching the web application, the machine learning pipeline must be executed.

Run:

```
</> Bash  
  
python train_model.py
```

This step performs the following tasks:

- Loads and preprocesses the dataset
- Trains all machine learning models
- Compares model performance
- Selects the best performing model
- Saves trained model artifacts
- Generates analytics charts
- Stores all required files inside the **model** folder

This process generally takes **30–60 seconds**.

STEP 5 — Launch the Flask Web Application :

Start the application using:

```
</> Bash  
  
python app.py
```

The terminal should display output similar to:

```
PS C:\Users\Yaswanth Ganesh\Desktop\SmartLender> python app.py  
* Serving Flask app 'app'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://10.94.106.188:5000  
Press CTRL+C to quit  
127.0.0.1 - - [01/Jul/2026 08:31:30] "GET / HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:31:34] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:31:43] "GET /static/js/main.js HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "POST /predict HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "GET /static/js/main.js HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /reports HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /static/js/main.js HTTP/1.1" 304 -
```

STEP 6 — Open the Application :

Visit:

```
http://127.0.0.1:5000
```

STEP 6 — (Optional) Running the Test Suite:

 **Bash**

```
python -m pytest tests/ -v
```

Exploring the Website's Web Pages:

Home Page:

LEDGER ENTRY • NEW APPLICATION

Screen a loan applicant

SL Smart Lender UNDERWRITING DESK

APPLICATION BATCH HISTORY REPORTS MODEL

1 APPLICANT PROFILE

Gender

Male Female

Marital Status

Married Single

Dependents

0

Education

Graduate Not Graduate

Self-Employed

No Yes

Property Area

Urban

2 INCOME & LOAN TERMS

Applicant Monthly Income

\$ 5000

Co-applicant Monthly Income

\$ 0

Requested Loan Amount

\$ 150

Loan Term

360 months (30 yr)

Credit History

✓ Meets guidelines

Model: Logistic Regression

RUN ELIGIBILITY CHECK

3 LIVE EMI CALCULATOR

MONTHLY PAYMENT

—

Total Repayment

—

Total Interest

—

Assumed Rate

8.5% p.a.

Updates live as you type. DTI = debt to income ratio.

4 SESSION STATISTICS

0 APPLICATIONS

0 APPROVAL RATE

0 AVE CREDITSCORE

5 ACTIVE MODEL

Best performer across 5 trained classifiers, selected automatically on held-out test accuracy.

Logistic Regression

86.18%

Decision Tree

83.74%

Random Forest

86.18%

KNN

84.55%

XGBoost

81.3%

6 HOW TO READ THE RESULT

A confidence score above 80% reflects strong agreement with historical approvals. Scores closer to 50% sit near the model's decision boundary and may warrant a manual second look.

Description: The Home Page serves as the primary interface of the Smart Lender application, allowing users to enter applicant details and obtain real-time loan eligibility predictions. It features a responsive loan application form where users provide personal, financial, and loan-related information, including income, employment status, loan amount, loan term, and credit history. The page also includes a live EMI calculator, session statistics, model performance summary, and guidance for interpreting prediction results. Designed with a clean, user-friendly layout, the home page enables users to efficiently submit applications and access key financial insights before performing the eligibility check.

Batch Prediction Page :

BATCH PROCESSING • BULK ELIGIBILITY SCREENING

Upload applicant CSV

Upload a CSV file containing multiple applicant rows and receive instant eligibility predictions for every record. Download the results as a ready-to-share report.

1 UPLOAD CSV FILE



Drag & drop your CSV here
or click to browse — .csv files only

No file selected

REQUIRED COLUMN NAMES

Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area

Same format as the training dataset. Any extra columns are preserved in the output.

RUN BATCH PREDICTION

2 CSV FORMAT GUIDE

Your file must have a header row with the column names shown. Values should match the training data exactly:

- Gender: Male or Female
- Married: Yes or No
- Dependents: 0, 1, 2, 3+
- Education: Graduate or Not Graduate
- Self_Employed: Yes or No
- ApplicantIncome: monthly income in dollars
- LoanAmount: amount in thousands
- Credit_History: 1(good) or 0(bad)
- Property_Area: Urban, Semiurban, Rural

3 TIPS

The output CSV will include three new columns:

- Predicted_Status: Y or N
- Approved: Yes or No
- Confidence_Pct: 0-100%

Description: The Batch Prediction page enables users to evaluate multiple loan applications simultaneously by uploading a CSV file containing applicant information. The interface provides a drag-and-drop file upload area, a detailed CSV format guide specifying the required columns, and usage tips to ensure correct file preparation. After uploading the dataset, the system processes each applicant using the trained machine learning model, generates loan eligibility predictions with confidence scores, and produces a downloadable CSV report containing the prediction results for all applicants. This feature significantly improves efficiency for banks and financial institutions handling large volumes of loan applications.

Charts and Analysis Section:



Description: The **Charts & Analysis** page provides a comprehensive overview of the performance of all machine learning models used in the Smart Lender system. It displays key evaluation metrics, including **Testing Accuracy**, **Cross-Validation Accuracy**, and **ROC-AUC Score** for Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. The best-performing model is clearly highlighted, allowing users to compare model performance and understand the basis for selecting the final prediction model. This page offers valuable insights into the effectiveness and reliability of each algorithm used during the training process.

Result Page :



Likely to repay



AI RISK FACTOR BREAKDOWN
How each feature influenced this prediction

Feature	Influence (%)
Credit History	47.05%
Marital Status	9.56%
Education Level	6.67%
Co-applicant Income	6.65%
Total Income (log)	6.16%
No. of Dependents	4.6%
Loan Amount	4.13%
Gender	3.85%

EST. MONTHLY EMI

\$3,157

DEBT-TO-INCOME RATIO

157.8%

APPLICATION SUMMARY

Gender	Male
Marital status	Yes
Dependents	2
Education	Graduate
Self-employed	No

LOANING DESK

APPLICATION	BATCH	HISTORY	REPORTS	MODEL
Loan amount	\$100k			
Loan term	36 months			
Credit history	✓ Meets guidelines			
Property area	Urban			

SUGGESTED NEXT STEPS

- Verify income documents and bank statements
- Conduct property valuation for collateral
- Final underwriter sign-off and loan disbursement

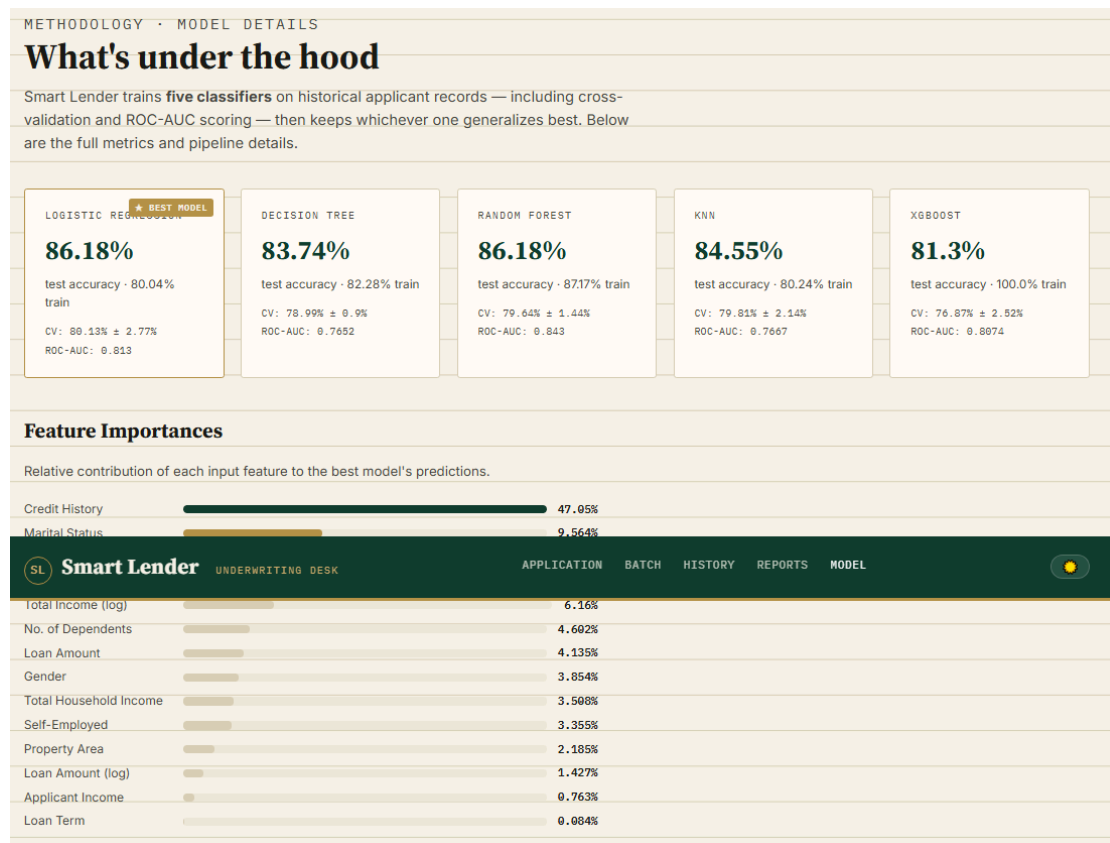
[SCREEN ANOTHER APPLICANT](#)

[VIEW HISTORY](#)

Description:

The **Prediction Result** page displays the outcome of the loan eligibility assessment after the applicant's information has been processed by the trained machine learning model. It presents the loan approval status along with a confidence score that indicates the model's prediction certainty. The page also includes an AI Risk Factor Breakdown, highlighting the contribution of each input feature to the prediction, enabling users to understand the factors influencing the decision. Additionally, it provides the estimated monthly EMI, Debt-to-Income (DTI) ratio, a comprehensive summary of the submitted application details, and suggested next steps for loan processing. This page offers a clear, transparent, and informative view of the prediction results, supporting better decision-making for both applicants and loan officers.

Model Details Page :



Description :

The **Model Details** page provides an overview of the machine learning models used in the Smart Lender system and their respective performance metrics. It compares the testing accuracy, training accuracy, cross-validation accuracy, and ROC-AUC scores of all implemented classifiers, highlighting the best-performing model selected for deployment. The page also presents the feature importance analysis, illustrating the relative contribution of each input feature to the model's predictions. This helps users understand the model selection process and the key factors influencing loan eligibility decisions, thereby improving transparency and interpretability of the AI system.

Conclusion:

The Smart Lender project successfully demonstrates how Artificial Intelligence and Machine Learning can modernize the traditional loan approval process by providing fast, consistent, and data-driven decision support.

By integrating advanced classification algorithms—including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost—the system evaluates applicant information and predicts loan eligibility with high accuracy. After comparative evaluation, **XGBoost** was selected as the final production model due to its superior predictive performance.

The application combines a Flask-based backend, responsive web interface, automated data preprocessing pipeline, real-time prediction engine, EMI and Debt-to-Income (DTI) calculators, RESTful APIs, and batch prediction capabilities into a single intelligent platform. These features enable banking professionals to process applications more efficiently, reduce manual effort, improve consistency in lending decisions, and minimize the risk of non-performing assets.

Overall, Smart Lender demonstrates the practical application of machine learning in the financial sector and provides a scalable foundation for future enhancements such as cloud deployment, user authentication, explainable AI, credit bureau integration, and continuous model retraining using newly collected loan data.